Assignment - 03

Unit 3: NoSQL Database (MongoDB) | BCA305T

Q1. Explain MongoDB as a NoSQL document-oriented database and compare its data model with traditional relational databases.

Answer:

MongoDB is an open-source, document-oriented NoSQL database. Instead of storing data in rows and columns of tables, it stores data as **documents** in **BSON** (Binary JSON) format. Documents are grouped into **collections**, and collections live inside a **database**. Each document is a set of field-value pairs, and values can be strings, numbers, arrays, or even nested documents. A collection does not force every document to have the same fields, so MongoDB is called *schema-flexible*.

```
{
  "_id": ObjectId("65a1f..."),
  "name": "Amit Sharma",
  "course": "BCA",
  "skills": ["Java", "MongoDB"],
  "address": { "city": "Roorkee", "pin": 247667 }
}
```

Comparison with relational databases

Aspect	Relational DB (e.g., MySQL)	MongoDB
Basic unit	Table with rows and columns	Collection with documents
Schema	Fixed, predefined; changes need ALTER TABLE	Dynamic; each document can differ
Relationships	Foreign keys and JOINs across tables	Embedding or references; $lookup when needed
Data format	Flat, normalized tuples	Nested JSON-like (BSON) documents
Scaling	Mainly vertical	Horizontal (sharding) plus vertical
Query language	SQL	MongoDB Query API (JSON-style)
Transactions	Strong ACID across tables	ACID per document; multi-document transactions supported
Best for	Structured data, complex joins	Semi-structured, evolving, large-scale data

Key idea: a relational design splits one real-world entity (e.g., an order with its items) across several tables, while MongoDB can keep the whole entity in a single document, which reduces joins and speeds up reads.

Q2. Explain the key features of document-oriented databases and discuss how MongoDB implements these features.

Answer:

A document-oriented database stores each record as a self-describing document (JSON, BSON or XML). Its main features and the way MongoDB implements them are:

Feature	Meaning	MongoDB implementation
Flexible schema	Documents in one collection may have different fields	No fixed schema; optional JSON Schema validation can be added
Rich data types	Nested objects and arrays	BSON supports arrays, embedded documents, dates, ObjectId, decimals, binary

Feature	Meaning	MongoDB implementation
Self-contained records	Related data kept together	Embedded documents avoid JOINs
Powerful querying	Query on any field, even nested	Query API, aggregation pipeline, text and geospatial search
Indexing	Fast lookups on fields	Single, compound, multikey, text, geospatial, TTL and hashed indexes
Scalability	Handle growing data and traffic	Sharding for horizontal scaling
High availability	Survive server failure	Replica sets with automatic failover
Horizontal distribution	Spread data over machines	Shard keys and balancer distribute chunks

Because of these features MongoDB maps naturally to objects used in application code, making development faster and easier to change over time.

Q3. Explain the concept of consistency in MongoDB. Discuss how MongoDB maintains data consistency in distributed environments.

Answer:

Consistency means every read returns the most recent successful write (or a state that is valid according to rules). In the CAP theorem MongoDB is generally classed as a **CP** system: when a network partition occurs it prefers consistency over availability for writes.

How MongoDB maintains consistency

- **Single primary:** in a replica set only one primary node accepts writes; secondaries copy its operations from the *oplog*. This avoids conflicting writes.
- **Write concern:** controls how many nodes must acknowledge a write. `w:1` means primary only; `w:"majority"` means a majority of members, so the write survives failover.
- **Read concern:** controls what data a read sees: *local*, *available*, *majority* (only majority-committed data), *linearizable* (latest majority-committed write) and *snapshot* (for transactions).
- **Read preference:** reading from the primary gives strongest consistency; reading from secondaries may return slightly stale data (eventual consistency).
- **Elections and rollback:** if the primary fails, a secondary with the latest data is elected. Writes not replicated to a majority are rolled back on the old primary when it rejoins.
- **Atomic single-document operations:** updates to one document, including its nested parts, are always atomic.
- **Causal consistency:** sessions can guarantee that reads reflect earlier writes in the same session.

```
db.orders.insertOne(
  { item: "Laptop", qty: 1 },
  { writeConcern: { w: "majority", j: true } }
)
db.orders.find().readConcern("majority")
```

Trade-off: stronger settings (majority, linearizable) give higher consistency but slightly higher latency; weaker settings give speed but may expose stale reads.

Q4. Explain transactions in MongoDB. How does MongoDB support transactions involving multiple operations and documents?

Answer:

A **transaction** is a group of operations that execute as one logical unit and satisfy the **ACID** properties: Atomicity (all or nothing), Consistency, Isolation and Durability. MongoDB always provided atomicity for a single document. From **version 4.0** it supports **multi-document ACID transactions** on replica sets, and from **4.2** on sharded clusters.

How it works

- A **session** is started and a transaction begins inside it.
- Operations (insert, update, delete) on one or more documents, collections or databases are executed inside the transaction.
- **commitTransaction** makes all changes visible together; **abortTransaction** discards all of them.
- Uses **snapshot isolation**: the transaction reads a consistent snapshot and does not see other uncommitted changes.
- Conflicting writes cause a *WriteConflict* error and the transaction can be retried. Transactions also have a time limit (60 seconds by default).

Example: money transfer

```
const session = db.getMongo().startSession();
const accounts = session.getDatabase("bank").accounts;
session.startTransaction({
  readConcern: { level: "snapshot" },
  writeConcern: { w: "majority" }
});
try {
  accounts.updateOne({ _id: "A" }, { $inc: { balance: -500 } });
  accounts.updateOne({ _id: "B" }, { $inc: { balance:  500 } });
  session.commitTransaction();
} catch (e) {
  session.abortTransaction();
} finally {
  session.endSession();
}
```

Good schema design (embedding related data in one document) reduces the need for transactions, because single-document updates are already atomic.

Q5. Discuss the availability features of MongoDB and explain how replication helps maintain high availability.

Answer:

Availability is the ability of a database to keep serving reads and writes even when hardware, network or server failures happen. MongoDB provides high availability mainly through **replica sets**.

Replica set

A replica set is a group of *mongod* instances holding the same data. It has:

- **Primary:** receives all write operations.
- **Secondaries:** copy the primary's oplog and apply it asynchronously; can serve reads if allowed.
- **Arbiter (optional):** takes part in elections but stores no data.

How replication gives high availability

- **Redundancy:** several copies of data exist on different servers, so one failure does not lose data.
- **Automatic failover:** members send heartbeats every 2 seconds; if the primary is unreachable, an election chooses a new primary, usually within seconds, without manual work.
- **Read scaling:** read preference such as *secondaryPreferred* spreads read load.

- **Backups and maintenance:** backups or upgrades can be done on a secondary with no downtime (rolling upgrades).
- **Hidden and delayed members:** hidden members serve reporting/backup; delayed members protect against human errors.
- **Geographic distribution:** members can be placed in different data centres to survive a site outage.
- **Retryable writes and reads:** drivers automatically retry once after a failover.

```
rs.initiate({ _id: "rs0", members: [
  { _id: 0, host: "node1:27017" },
  { _id: 1, host: "node2:27017" },
  { _id: 2, host: "node3:27017" } ] })
```

A set should have an odd number of voting members (at least 3) so that a majority can always be formed.

Q6. Explain the query features of MongoDB with suitable examples of CRUD operations and filtering documents.

Answer:

MongoDB offers a rich query API using JSON-like filter documents. Features include field and nested-field queries, comparison and logical operators, array queries, regular expressions, projection, sorting, limiting, indexing, text search, geospatial queries and the aggregation framework.

Create

```
db.students.insertOne({ name: "Riya", age: 20, course: "BCA", marks: 82 })
db.students.insertMany([
  { name: "Aman", age: 21, course: "BCA", marks: 74 },
  { name: "Neha", age: 22, course: "MCA", marks: 91 } ])
```

Read and filtering

```
db.students.find()                              // all documents
db.students.find({ course: "BCA" })             // equality
db.students.find({ marks: { $gt: 80 } })        // comparison
db.students.find({ $or: [ {course:"MCA"}, {age:{$lt:21}} ] })
db.students.find({ course: {$in: ["BCA","MCA"]} }, { name:1, marks:1, _id:0 })
db.students.find({ name: /^A/ })                // regex
db.students.find().sort({ marks: -1 }).limit(3)
db.students.find({ "address.city": "Roorkee" }) // nested field
db.students.find({ skills: "Java" })            // array match
```

Update

```
db.students.updateOne({ name: "Riya" }, { $set: { marks: 88 } })
db.students.updateMany({ course: "BCA" }, { $inc: { marks: 2 } })
db.students.updateOne({ name: "Aman" }, { $push: { skills: "Python" } })
```

Delete

```
db.students.deleteOne({ name: "Neha" })
db.students.deleteMany({ marks: { $lt: 40 } })
```

Aggregation example

```
db.students.aggregate([
  { $group: { _id: "$course", avgMarks: { $avg: "$marks" } } },
  { $sort: { avgMarks: -1 } } ])
```

Operator	Use
$eq, $ne, $gt, $gte, $lt, $lte	Comparison
$and, $or, $not, $nor	Logical

Operator	Use
$in, $nin	Match in a list
$exists, $type	Field presence and type
$set, $inc, $push, $pull, $unset	Update operators

Q7. Explain MongoDB scaling mechanisms. Differentiate between vertical scaling and horizontal scaling with reference to MongoDB.

Answer:

Scaling means increasing a database's capacity to handle more data and more requests. MongoDB supports both vertical and horizontal scaling.

Vertical scaling (scale up)

Adding more resources (CPU, RAM, faster SSD) to a single server. It is simple and needs no change in the application, but it is limited by hardware, becomes costly and leaves a single point of failure.

Horizontal scaling (scale out)

Distributing data across multiple servers using **sharding**. A sharded cluster has:

- **Shards:** each holds a subset of the data (each shard is normally a replica set).
- **mongos routers:** receive client queries and route them to the right shard(s).
- **Config servers:** store cluster metadata and chunk-to-shard mapping.
- **Shard key:** the field(s) used to split data into chunks; can be *ranged* or *hashed*. The balancer moves chunks to keep shards even.

```
sh.enableSharding("shop")
sh.shardCollection("shop.orders", { customerId: "hashed" })
```

Point	Vertical scaling	Horizontal scaling
Method	Bigger single machine	More machines (shards)
MongoDB feature	Upgrade server hardware	Sharding (plus replica sets)
Limit	Hardware ceiling	Practically unlimited
Cost	Expensive at high end	Uses commodity hardware
Fault tolerance	Single point of failure	Failure affects only one shard
Complexity	Low	Higher (shard key design, routing)
Downtime	Often needed for upgrades	Shards can be added online

Q8. Discuss the suitable use cases of MongoDB and explain why document databases are preferred for applications with flexible data structures.

Answer:

Suitable use cases

- **Content management systems** and blogging platforms.
- **E-commerce** product catalogues with varying attributes.
- **Event logging** and application monitoring.
- **Real-time analytics** and web analytics.
- **Mobile and web apps** with rapidly changing requirements.

- **Internet of Things (IoT)** and time-series sensor data.
- **Personalization and user profiles**, social networks, gaming leaderboards.
- **Catalogues and single-view applications** that combine data from many sources.

Why document databases suit flexible structures

- **No rigid schema:** new fields can be added to new documents without migrating the whole collection.
- **Natural mapping:** JSON documents match objects in programming languages, reducing object-relational mismatch.
- **Heterogeneous data:** products such as a phone and a shirt can have different attributes in the same collection.
- **Nested data and arrays:** complex entities are stored together and fetched in one read.
- **Faster development:** supports agile and iterative changes.
- **Easy scale-out:** sharding handles growing data volumes.

Not ideal for: highly relational data needing many complex joins or strict multi-table integrity, where a relational DB is usually better.

Q9. Explain how MongoDB can be used for event logging. Design a suitable document structure for storing application or system events.

Answer:

Applications generate large numbers of events (errors, logins, API calls, system alerts). MongoDB suits event logging because:

- Writes are fast and can be scaled across shards.
- Different event types can have different fields in the same collection (flexible schema).
- **Capped collections** or **TTL indexes** automatically remove old logs.
- Aggregation can summarize events by time, type, severity or source.
- Time series collections (MongoDB 5.0+) store timestamped data efficiently.

Sample event document

```
{
  "_id": ObjectId("..."),
  "timestamp": ISODate("2026-10-30T09:15:22Z"),
  "level": "ERROR",                  // INFO, WARN, ERROR, CRITICAL
  "eventType": "payment_failed",
  "source": { "app": "checkout-service", "host": "srv-12", "env": "prod" },
  "user": { "userId": 1045, "ip": "103.21.4.9" },
  "message": "Card declined",
  "details": { "orderId": "ORD-8891", "amount": 2499, "gateway": "XPay" },
  "tags": ["payment", "retry"]
}
```

Useful commands

```
// auto-delete after 30 days
db.events.createIndex({ timestamp: 1 }, { expireAfterSeconds: 2592000 })
// index for fast filtering
db.events.createIndex({ level: 1, eventType: 1, timestamp: -1 })
// capped collection (fixed size, oldest overwritten)
db.createCollection("syslog", { capped: true, size: 104857600 })
// errors per type
db.events.aggregate([ { $match: { level: "ERROR" } },
```

```
  { $group: { _id: "$eventType", count: { $sum: 1 } } } ])
```

Q10. Discuss the use of MongoDB in Content Management Systems (CMS). Explain how its document model supports dynamic and heterogeneous content.

Answer:

A CMS stores many kinds of content such as articles, pages, images, videos, products, comments and metadata. Each type has different attributes, and editors often add new fields. MongoDB is a strong fit because of its document model.

How the document model helps

- **Heterogeneous content:** articles, galleries and videos can live in one collection, each with its own fields.
- **Dynamic schema:** adding a new field (e.g., *seoTitle*) needs no table alteration.
- **Nested and array data:** author details, tags, categories, image lists and revisions are embedded in the same document.
- **Versioning:** revision history can be kept as an array or in a separate collection.
- **Fast page loading:** one read returns a full page with its content blocks.
- **Search and indexing:** text indexes allow full-text search over content.
- **Scalability:** sharding and replication handle high traffic news and media sites.

Example documents in one collection

```
{ "type": "article", "title": "Intro to NoSQL", "author": "Ravi",
  "body": "...", "tags": ["db","nosql"], "publishedAt": ISODate("2026-10-01") }

{ "type": "video", "title": "MongoDB Demo", "url": "https://...",
  "duration": 540, "resolutions": ["480p","720p","1080p"] }

{ "type": "gallery", "title": "Campus Day",
  "images": [ {"src":"a.jpg","caption":"Entry"}, {"src":"b.jpg","caption":"Stage"} ] }
```

Popular CMS platforms and headless CMS tools often use MongoDB for these reasons.

Q11. Explain the role of MongoDB in blogging platforms. Design a suitable MongoDB document structure for managing users, posts, comments, and tags.

Answer:

A blogging platform handles users, posts, comments, likes and tags. MongoDB helps because posts and their comments are naturally hierarchical, the content structure changes often, and read traffic is high. A single query can fetch a post together with its author details and tags.

Design (collections)

```
// users
{ "_id": ObjectId("u1"), "username": "riya_dev", "email": "riya@mail.com",
  "passwordHash": "...", "bio": "Writer", "role": "author",
  "createdAt": ISODate("2026-01-10") }

// posts
{ "_id": ObjectId("p1"), "title": "Learning MongoDB", "slug": "learning-mongodb",
  "content": "....", "authorId": ObjectId("u1"),
  "author": { "username": "riya_dev" },          // embedded copy for fast display
  "tags": ["mongodb", "nosql", "database"],
  "status": "published", "views": 1520, "likes": 87,
  "createdAt": ISODate("2026-10-05"),
  "recentComments": [ { "user": "aman", "text": "Nice!" } ] }

// comments
```

```
{ "_id": ObjectId("c1"), "postId": ObjectId("p1"), "userId": ObjectId("u2"),
  "text": "Very helpful post", "parentId": null,
  "createdAt": ISODate("2026-10-06") }

// tags
{ "_id": "mongodb", "description": "MongoDB articles", "postCount": 42 }
```

Design notes

- **Embed** data that is read together and small (tags array, author name, few recent comments).
- **Reference** data that grows without bound (all comments) to avoid the 16 MB document limit.
- Indexes: `posts.slug` (unique), `posts.tags`, `comments.postId`, and a text index on title and content.

```
db.posts.find({ tags: "mongodb", status: "published" }).sort({ createdAt: -1 })
db.comments.find({ postId: ObjectId("p1") }).sort({ createdAt: 1 })
```

Q12. Discuss the use of MongoDB in web analytics and real-time analytics applications. Explain how its query and scalability features support such applications.

Answer:

Web analytics systems collect page views, clicks, sessions, referrers and device details at high speed and must show up-to-date dashboards. MongoDB is used for this because it can ingest large streams of data and query them quickly.

Support from query features

- **Atomic update operators** such as `$inc` allow real-time counters (page views, clicks) with upsert.
- **Aggregation pipeline** groups, filters and summarizes data by page, time, country or device.
- **Change streams** push live changes to dashboards or other services.
- **Compound and TTL indexes** make time-range queries fast and remove old raw data.
- **Time series collections** store timestamped metrics compactly with fast queries.
- **Flexible schema** lets new tracked attributes be added without migration.

Support from scalability features

- **Sharding** by a key such as site ID or hashed visitor ID spreads heavy write load across servers.
- **Replica sets** give availability and allow analytical reads on secondaries without disturbing writes.
- **In-memory working set** (WiredTiger cache) provides low-latency reads.

```
// real-time counter
db.pageStats.updateOne(
  { page: "/home", day: "2026-10-30" },
  { $inc: { views: 1 } }, { upsert: true })

// top pages today
db.pageviews.aggregate([
  { $match: { ts: { $gte: ISODate("2026-10-30") } } },
  { $group: { _id: "$page", hits: { $sum: 1 } } },
  { $sort: { hits: -1 } }, { $limit: 5 } ])
```

Q13. Explain how MongoDB can be used in E-Commerce applications. Discuss suitable collections for products, customers, orders, and shopping carts.

Answer:

E-commerce sites have product catalogues with very different attributes, heavy traffic, and frequent changes. MongoDB handles varied product data, fast catalogue reads, scalable order storage and personalization.

Collections

```
// products - attributes differ by category
{ "_id": "P100", "name": "Smartphone X", "category": "Mobiles", "price": 19999,
  "stock": 120, "attributes": { "ram": "8GB", "storage": "128GB", "color": "Black" },
  "images": ["x1.jpg","x2.jpg"], "rating": 4.4 }
{ "_id": "P200", "name": "Cotton Shirt", "category": "Clothing", "price": 799,
  "attributes": { "size": ["M","L","XL"], "fabric": "Cotton" } }

// customers
{ "_id": "C1", "name": "Sneha", "email": "sneha@mail.com",
  "addresses": [ { "type": "home", "city": "Aligarh", "pin": 202001 } ],
  "wishlist": ["P100"] }

// orders - snapshot of price and items at purchase time
{ "_id": "O5001", "customerId": "C1", "orderDate": ISODate("2026-10-29"),
  "items": [ { "productId": "P100", "name": "Smartphone X", "qty": 1, "price": 19999 } ],
  "total": 19999, "status": "shipped",
  "shipping": { "city": "Aligarh", "pin": 202001 },
  "payment": { "method": "UPI", "status": "paid" } }

// carts
{ "_id": "CART1", "customerId": "C1",
  "items": [ { "productId": "P200", "qty": 2 } ],
  "updatedAt": ISODate("2026-10-30") }
```

Why it works well

- Different product types sit in one *products* collection with a flexible *attributes* object.
- Orders embed items, so an entire order is read or written atomically in one document.
- Indexes on category, price and text support filtering and search; carts can use a TTL index to expire abandoned carts.
- Multi-document transactions can be used to reduce stock and create the order together.
- Sharding scales the catalogue and orders during sales seasons.

Q14. Explain the challenges of handling complex transactions that span different operations in MongoDB. Discuss how MongoDB transactions can address these challenges.

Answer:

Challenges

- **Partial failure:** if one operation succeeds and the next fails (e.g., stock reduced but order not created), data becomes inconsistent.
- **Multiple documents or collections:** only single-document writes are atomic by default, so related updates in different documents are not atomic.
- **Concurrency:** several users updating the same data can cause lost updates or dirty/inconsistent reads.
- **Distributed systems:** operations may touch several shards or nodes, with network failures and failovers in between.
- **Durability and rollback:** the system must undo changes safely on error.

- **Manual workarounds:** before 4.0, developers used two-phase commit patterns in application code, which was complex and error-prone.

How MongoDB transactions address them

- **Atomicity:** all operations commit together or none do (abort rolls back everything).
- **Isolation:** snapshot isolation gives a consistent view and prevents dirty reads; write conflicts are detected and can be retried.
- **Consistency across documents, collections, databases and shards** (distributed transactions from 4.2).
- **Durability:** with `w:"majority"` committed data survives failover.
- **Retryable transaction errors:** the *TransientTransactionError* label tells the application it is safe to retry; the driver's *withTransaction()* helper automates this.

```
await session.withTransaction(async () => {
  await products.updateOne({ _id: "P100" }, { $inc: { stock: -1 } }, { session });
  await orders.insertOne({ item: "P100", qty: 1, status: "placed" }, { session });
});
```

Best practice: keep transactions short, design the schema so most updates touch a single document, and use transactions only when multi-document atomicity is really needed, because they add overhead.

Q15. Explain how MongoDB supports queries against varying aggregate structures. Discuss the advantages of its flexible schema with suitable examples.

Answer:

An **aggregate** is a collection of related data stored and handled as one unit (for example, a customer with addresses, or an order with items). In MongoDB each aggregate is a document, and different documents in the same collection may have different structures. MongoDB still lets you query all of them with one query API.

Querying varying structures

```
// Collection "products" with different structures
{ "name": "Laptop", "ram": "16GB", "ports": ["USB","HDMI"] }
{ "name": "T-Shirt", "size": ["M","L"], "color": "Blue" }
{ "name": "Book", "author": "R. K. Narayan", "pages": 220 }

db.products.find({ ram: "16GB" })               // only docs having 'ram'
db.products.find({ size: { $exists: true } })   // docs that have 'size'
db.products.find({ pages: { $gt: 200 } })
db.products.find({ $or: [ {color:"Blue"}, {ram:"16GB"} ] })
db.products.find({ ports: "HDMI" })             // array match
db.products.find({ pages: { $type: "int" } })   // by data type
```

Fields missing from a document simply do not match a filter, so queries run safely across mixed structures. Dot notation (`"specs.cpu.cores"`) reaches nested fields, and sparse or partial indexes can index fields that exist only in some documents. Optional **JSON Schema validation** can enforce rules where needed.

Advantages of flexible schema

- **Easy evolution:** new fields can be added without altering the structure or taking the database offline.
- **Supports polymorphic data:** different types (products, content, devices) live in one collection.
- **Faster development:** no migrations for every change; suits agile teams.
- **Natural modelling:** data matches application objects, reducing mapping effort.

- **No empty NULL columns:** only relevant fields are stored, saving space.
- **Fewer joins:** related data embedded in a single aggregate gives faster reads.
- **Data integration:** data from different sources with different formats can be stored together.

Caution: too much freedom can lead to inconsistent data, so teams should use schema validation, naming conventions and indexes where appropriate.